

서찬진 · ChanJin Seo

AI Agent Backend · AI Platform · AX · DevOps

AI Backend · AI Platform Engineer

AI 기능을 상태·실패·배포·관측성까지 연결해 운영 가능한 서비스로 구현합니다.

01

AI Pipeline & Backend

RAG · Embedding · Async Processing State Management · Failure Handling

02

Platform & Operations

Docker · K3s · GitHub Actions Prometheus · Grafana · Troubleshooting

03

AX & Automation

Workflow Design · Human Approval Agent-assisted Development

AI 서비스 설계·구현

3-Node Hybrid K3s 운영

로그·지표 기반 검증과 개선

Email scj1507@gmail.com

GitHub github.com/seochanjin

프로젝트 요약 및 목차

문제 정의부터 설계·구현·운영·회고까지 이어진 3개의 프로젝트를 정리했습니다.

01 NewsLab 2026.05–Present · Personal Project

상세 페이지
03–08

한 줄 설명

여러 뉴스 기사를 Topic 단위로 묶고 요약해 제공하는 장기 운영 AI 뉴스 플랫폼

본인 역할

AI Pipeline · Backend API · K3s Operations Monitoring · Troubleshooting

대표 성과

- RSS 10개 Source에서 기사 4,500여 건 수집
- 고비용 원문 추출 범위를 전체 기사의 7.5%로 제한

운영 지표 기준: 2026.07.07

FastAPI · PostgreSQL/pgvector · Next.js · Docker · K3s · GitHub Actions · Prometheus/Grafana

AI Backend

K3s 운영

관측성

02 NotebookLM Clone 2026.03–2026.04 · Personal Project

상세 페이지
09–14

한 줄 설명

PDF 분석부터 Hybrid RAG 답변과 근거 저장까지 구현한 문서 기반 AI 서비스

본인 역할

Spring Boot–FastAPI 책임 분리 문서 상태 · Reference · AiCallLog · RAG Pipeline 설계

대표 성과

- PROCESSING · COMPLETED · FAILED 기반 문서 상태 관리
- pgvector MMR · Keyword Search · Local Reranker 결합
- Quota 최종 실패 시 Partial Vector Cleanup 적용

Spring Boot · FastAPI · PostgreSQL/pgvector · Redis · Gemini API · Docker Compose · Nginx

RAG

처리 상태 관리

실패 복구

03 ACCIO 2025.10–2025.11 · 5인 Team Project

상세 페이지
15–18

한 줄 설명

자연어 상품 검색과 대화형 탐색 경험을 구현한 AI 쇼핑 서비스

본인 역할

Frontend Routing · State Management AI 검색 상태 복원 · Guest Cart 병합 · 인증 Flow 개선

대표 성과

- 화면 이동·새로고침 이후 AI 대화와 검색 결과 복원
- Redis 장애 시 Process Memory Fallback 적용
- 비회원 Cart와 로그인 사용자 Cart 병합

React · TypeScript · FastAPI · Elasticsearch · MongoDB · Redis · AWS Bedrock

AI UX

상태 복원

세션 관리

여러 기사를 하나의 뉴스 흐름으로 재구성하는 AI 플랫폼

기간

2026.05~2026.07
이후 운영/유지보수

형태

개인 프로젝트 실제 운영

Live Service

<https://newslab.ai.kr>

GitHub

[backend](#)
[frontend](#)

기획 배경

K3s를 단순 설치 실습으로 끝내지 않고, 정기적인 데이터 수집·Batch 처리·API·Web 배포와 장애 대응을 반복해서 경험할 수 있는 장기 프로젝트로 시작했습니다.

문제 정의

RSS 기사 목록만으로는 여러 매체가 반복 보도하는 같은 사건을 구분하기 어렵고, 하루의 주요 뉴스 흐름을 빠르게 파악하기 어렵습니다.

핵심 가설

제목과 RSS 요약에 Embedding해 의미적으로 유사한 기사를 묶고, 반복 보도량을 주목도 신호로 활용하면 기사를 Topic 단위 뉴스 흐름으로 재구성할 수 있다고 판단했습니다.

목표

선정된 기사에만 원문 추출과 LLM 요약에 수행해 비용과 실패 범위를 제한하면서 Daily·3일·주간 뉴스 흐름을 제공합니다.

기술 스택

Platform

Docker · K3s · Traefik · GitHub Actions · Prometheus/Grafana

Backend/Data

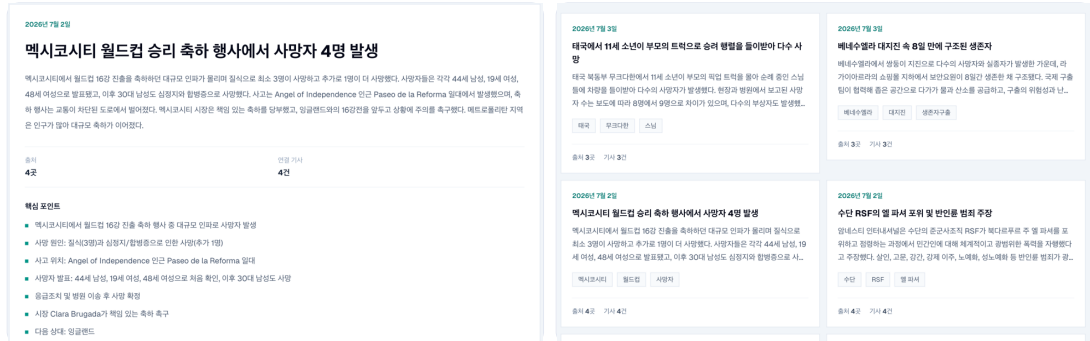
Python · FastAPI · PostgreSQL/pgvector

Frontend

Next.js · TypeScript



홈 화면 · 주요 이슈 · 최근 흐름

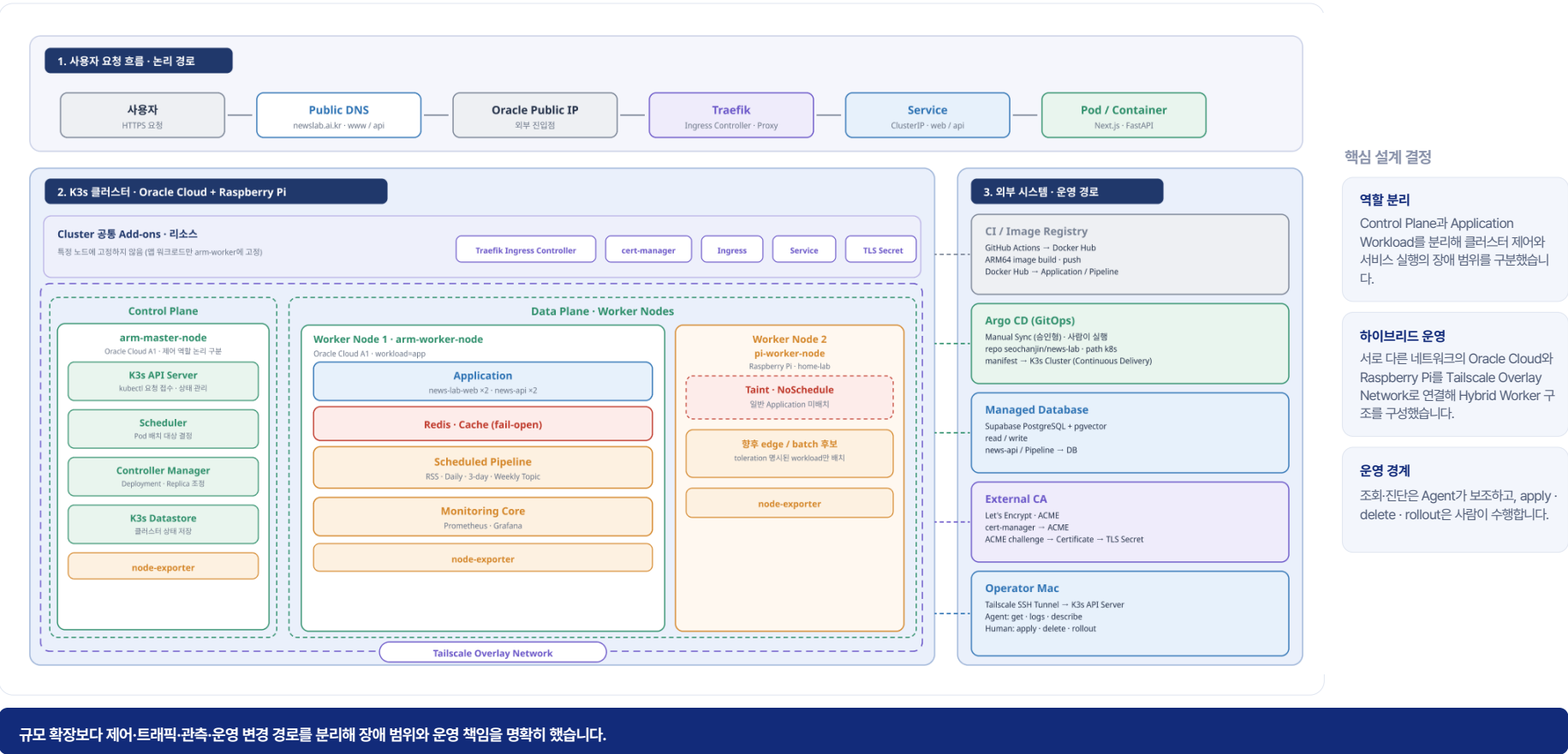


상세 페이지 · 핵심 포인트 · 키워드

토픽 페이지 · 관련 기사 모음

Hybrid K3s 위에서 운영하는 End-to-End 구조

Oracle Cloud A1 노드 2대와 Raspberry Pi Worker 1대를 Tailscale 기준으로 통합 운영합니다.



모든 기사에 고비용 처리를 하지 않는 단계형 AI Pipeline

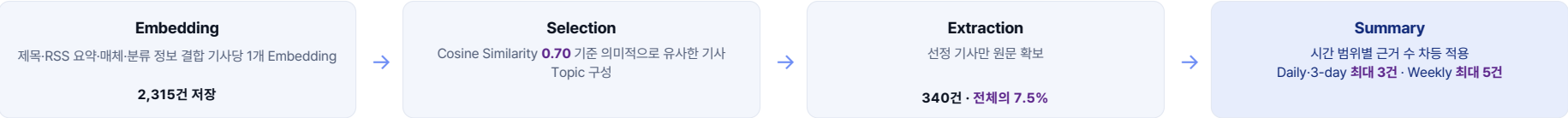
문제

- 모든 기사 원문을 선추출하면 외부 요청 수와 실패 지점이 기사 수에 비례해 증가
- Daily·3-day·Weekly 실행마다 Embedding을 다시 만들면 동일 기사에 비용과 처리시간이 반복 발생
- 기사 단위 오류를 격리하지 않으면 일부 실패 때문에 해당 실행 전체 결과가 영향받을 수 있음

설계 결정

- 제목·RSS 요약·매체·분류 정보를 결합해 기사당 1개 Embedding을 생성하고 Source Hash로 재사용
- 운영 기사 샘플에서 Cosine Similarity Threshold 0.60~0.80을 비교해 운영값 **0.70** 선택
- 시간 범위별 기사량에 따라 Summary 근거 수를 차등 적용 Daily·3-day **최대 3건** · Weekly **최대 5건**
- Stage별 실패 격리로 부분 실패가 전체 Batch로 번지지 않게 설계

Stage 분리 — 전체 기사에서 근거까지, 실패 격리 단위



수집 4,512건 → Embedding 2,315건 → 원문 추출 340건(7.5%) → Summary 135회

운영 지표 기준: 2026.07.07

Daily CronJob 산출물을 3-day · Weekly가 재사용

Daily CronJob · 04:00 매일 실행

기사 메타데이터 Embedding (DB 저장) → Selection → Extraction → Summary

Daily Summary 근거 최대 3건

3-day·Weekly는 기존 Embedding을 재사용하고 Selection · Extraction · Summary 단계만 다시 실행합니다.

3-day 매일 실행

최근 72시간 기사를 기존 Embedding으로 재클러스터링 3-day 전용 테이블과 API에 저장 Summary 근거 최대 3건

Weekly 주 1회 실행

직전 완료 주간 기사를 기존 Embedding으로 재클러스터링 Weekly 전용 테이블과 API에 저장 Summary 근거 최대 5건

NewsLab 운영 장애 진단 및 대응

Networking · Batch · Memory 세 가지 사례의 증상, 판단, 조치, 검증을 정리했습니다.

Networking

Ready인데 kubectl logs 실패

상황

유휴 Raspberry Pi를 K3s Worker로 추가해 Cloud와 Home Lab을 결합한 Hybrid Cluster 구성

문제

OCI Private Network와 가정 LAN의 경계가 다르고 Node가 Private-LAN-Tailscale 주소를 혼용

판단·조치

소규모 3-Node 환경에서 설정이 단순한 Tailscale을 선택하고 node-ip / flannel-iface를 Tailscale 주소로 통일

검증

양쪽 Worker 로그 확인 Pi에 NoSchedule Taint 적용

Evidence: docs/devlog/infra-pi-worker-join.md

Batch

Daily Pipeline 5시간 이상 Running

상황

Daily Pipeline이 5시간 이상 Running 상태로 남음

한계

재실행에서는 정상 완료되어 동일 현상을 재현하지 못했고 확정 원인은 특정하지 못함

조치

concurrencyPolicy: Forbid activeDeadlineSeconds: 1800 backoffLimit: 1 Stage Progress Log 추가

효과

중복 실행과 장시간 실행의 상한을 설정하고 재발 시 정체 Stage를 확인할 진단 경로 확보

Evidence: k8s/news-daily-topic-pipeline-cronjob.yaml

Memory

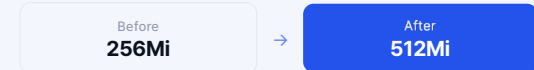
Grafana OOMKilled · Exit 137

배경

제한된 Worker 자원에서 Monitoring Core를 운영하고 Prometheus 데이터 보관 기간을 1일로 제한

판단·조치

Container Memory Limit 초과로 판단



검증

3/3 Running · Restart 0

Evidence: k8s/monitoring/kube-prometheus-stack-values.yaml

관측 가능한 증상과 실패 범위를 분리하고, 원인이 불명확한 경우에도 운영 피해의 상한과 재진단 경로를 먼저 설계했습니다.

AI Agent 개발 작업의 승인·검증 경계 설계

Agent의 구현과 Production 반영 사이에 Review · Human Approval · Verification 경계를 두었습니다.

AI AGENT DEVELOPMENT HARNESS

01 Human Define

요구사항 · 완료 기준 · 금지 영역 정의



02 Agent Work

승인된 범위에서 구현 코드 · 테스트 · 문서 수정



03 Review & Human Approval

Antigravity · CodeRabbit 독립 Review 사람이 수정 범위를 승인

Review Issue → Human Approval → Agent Rework → Verification



04 Verification → Human Production

허용된 테스트와 기존 계약 검증 Merge · DB Migration · K3s 반영은 사람이 수행

01 시작 계기

처음에는 AI를 검색과 코드 보조 도구로 사용했습니다. 이후 개발 과정 전반에 AI를 활용하는 사례를 접하면서, 단순 질의가 아니라 반복 가능한 개발 Workflow로 구성하는 방법을 고민했습니다.

02 적용 방향

Harness Engineering의 권한·검증·상태 관리 개념을 참고해 Agent의 작업 범위, 독립 Review, 사람 승인, Production 변경 권한을 분리했습니다. Review 결과를 자동 반영하지 않고, 사람이 승인한 항목만 Agent가 다시 수정하도록 구성했습니다.

03 실제 적용

Daily Topic Pipeline을 1개 진입점에서 **4개** Stage로 분리했습니다 — Embedding · Topic Selection · Raw Acquisition · Summary·Save. Review 과정에서 각 Stage가 날짜를 개별 계산하면 UTC/KST 경계에서 서로 다른 pipeline_date를 사용할 가능성을 발견했습니다. 공통 PipelineContext를 도입해 모든 Stage에 동일한 날짜를 전달했습니다. Stage 분리 작업 당시 날짜 경계 테스트를 포함해 전체 **150개** 테스트를 통과했고, API·DB Schema 및 금지 영역 변경 **0건**으로 기존 계약을 유지했습니다.

04 배운 점과 한계

Agent를 많이 사용하는 것보다 어떤 작업을 맡기고 어느 지점에서 사람이 판단할지를 정의하는 것이 더 중요했습니다. 다만 작업 상태가 여러 문서에 분산돼 세션이 현재 상태를 한 번에 파악하기 어렵습니다. 향후 단일 PROGRESS.md와 기계 가독 Feature 상태를 추가해 Session Lifecycle을 보완할 계획입니다.

Agent — 구현 · 테스트 · 문서 · 검증 자료 작성

Human — 요구사항 확정 · 수정 범위 승인 · Merge · DB/K3s/Secret 변경 · 운영 검증

NewsLab / 운영 리뷰와 성장 방향

이전 프로젝트 대비 개선한 점, 운영에서 얻은 판단, 남은 과제와 다음 성장 방향을 정리했습니다.

01 전 프로젝트 대비 개선점

1. 서비스 범위 확장

NotebookLM Clone이 문서 기반 AI 서비스 구현에 집중했다면, NewsLab은 수집·AI 처리·API·Web·배포·모니터링까지 실제 운영 범위로 확장했습니다.

2. 실행 환경 확장

Docker Compose 중심 환경에서 Oracle Cloud와 Raspberry Pi를 연결한 3-Node Hybrid K3s로 확장했습니다.

3. 처리 방식 확장

요청 단위 처리에서 Daily·3-day·Weekly CronJob 기반 Batch Pipeline으로 확장하고, Embedding 재사용과 선택 원문 추출을 적용했습니다.

4. 개발 Workflow 확장

Agent를 코드 보조가 아니라 독립 Review, Human Approval, Verification이 포함된 반복 가능한 Workflow에 포함했습니다.

기능 구현 중심 프로젝트에서 운영·자동화·관측성을 함께 다루는 프로젝트로 확장했습니다.

03 남은 한계와 다음 개선

1 최소 Alerting과 SLO

현재: Prometheus·Grafana로 상태 확인은 가능하지만, 장애 발생을 직접 확인해야 합니다.
다음: CronJob 실패, Node NotReady, Pod Restart에 대한 Alert Rule을 구성합니다.

2 Backup / Restore 검증

현재: 운영 데이터와 실행 이력은 저장하지만, Restore 훈련은 수행하지 않았습니다.
다음: Backup 정책과 함께 Restore 절차와 복구 검증 기록을 구성합니다.

3 운영 파라미터 재검증

현재: Cosine Similarity 0.70, Daily·3-day 최대 3건, Weekly 최대 5건을 사용합니다.
다음: 소스 수와 Topic 크기가 증가하면 임계값과 근거 수를 재검증합니다.

운영 위험을 줄이는 변경부터 우선순위를 두고 개선할 계획입니다.

02 운영하며 얻은 인사이트

1. 쿠버네티스 운영

Manifest 작성보다 Control Plane, Node, Pod, Network 경로를 나눠 장애 범위를 좁히는 능력이 더 중요했습니다.

2. AI 파이프라인

Model과 Prompt뿐 아니라 처리 상태, 부분 실패, 재실행, 날짜 경계, 저장 정합성까지 함께 설계해야 했습니다.

3. 에이전트 작업 흐름

작업을 많이 맡기는 것보다 수정 범위, 검증 기준, 운영 권한을 어디서 사람이 통제할지 정하는 것이 중요했습니다.

4. 운영 판단

구현한 선택을 무조건 방어하기보다 실제 운영 결과와 장애 기록을 기준으로 다시 평가해야 했습니다.

운영은 기능이 동작하는지 확인하는 일이 아니라, 실패 범위와 복구 경로를 설계하는 일이었습니다.

04 다음 성장 방향

1. 연결된 시각

NewsLab을 통해 AI 백엔드, 플랫폼, 업무 자동화가 하나의 서비스 안에서 연결된다는 점을 확인했습니다.

2. AI 백엔드

AI 기능 자체보다 처리 상태, 실패 계약, 도구 실행, 권한과 감사 가능성까지 함께 설계하고 싶습니다.

3. 플랫폼 운영

GitOps, SLI·SLO, Alerting, Restore 검증을 통해 운영 신뢰성을 높이는 경험을 확장하고 싶습니다.

4. 개발 방식의 균형

Agent는 반복 작업과 Review에 활용하고, 핵심 로직은 직접 설계·구현해 생산성과 코드 이해도를 함께 확보하겠습니다.

AI 기능을 데이터 흐름·운영 환경·권한 경계까지 연결하는 AI 백엔드 / 플랫폼 엔지니어로 성장하고자 합니다.

문서 근거와 처리 상태를 관리하는 RAG 서비스

PDF 업로드, 비동기 분석, Hybrid RAG 검색, 답변 근거 저장을 하나의 서비스 흐름으로 구현했습니다.

기간
2026.03~2026.04

형태
개인 프로젝트
4개 Repository 분리 구성
배포검증 NCP ·

GitHub
<https://github.com/notebooklm-clone-sci>

기획 배경

팀 프로젝트에서 경험한 RAG를 직접 설계·구현하며, Java-Spring 기반 서비스 관리와 Python-FastAPI 기반 AI 처리 구조를 학습하기 위해 시작했습니다.

문제 정의

PDF 분석은 처리 시간이 길고 외부 API 실행 시 문서 상태와 Vector 데이터가 어긋날 수 있었습니다. 또한 답변 근거를 저장하지 않으면 이후 어떤 문서를 기반으로 답변했는지 다시 확인하기 어려웠습니다.

설계 방향

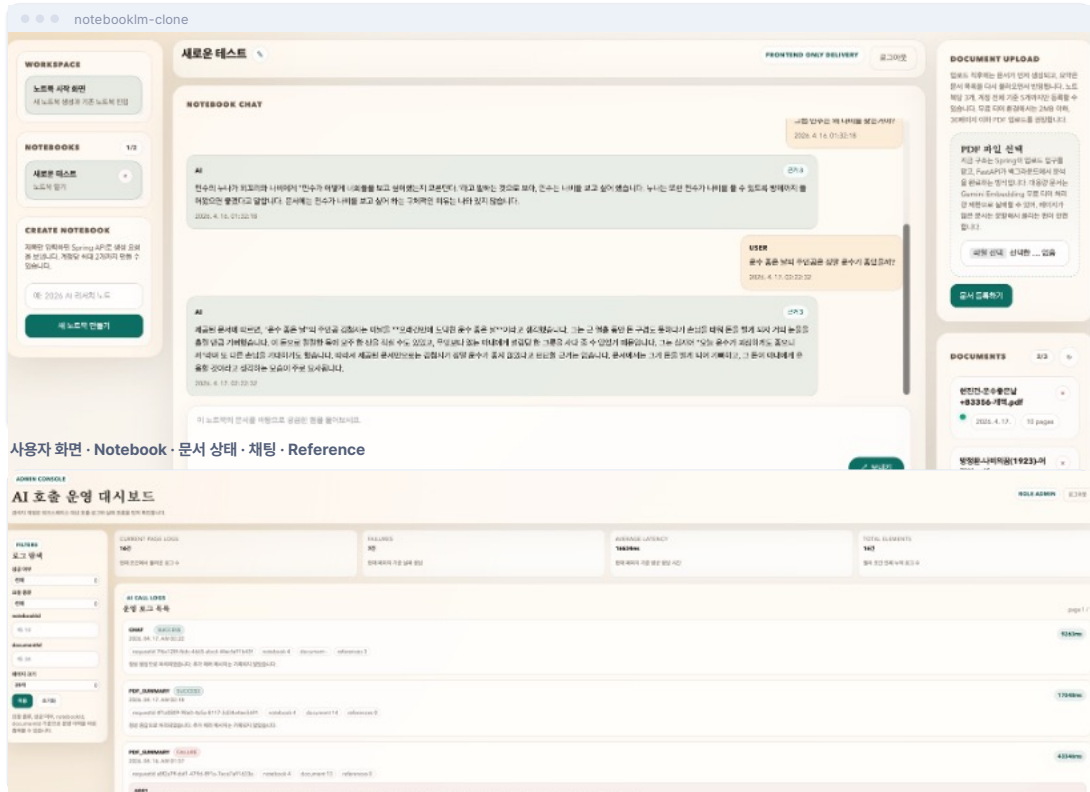
Spring Boot는 사용자·문서·채팅·처리 상태를 관리하고, FastAPI는 PDF 파싱·Embedding·검색·LLM 호출을 담당하도록 책임을 분리했습니다. 문서 분석은 비동기로 실행하고 결과를 PROCESSING · COMPLETED · FAILED 상태로 관리했습니다.

핵심 구현

Hybrid RAG 검색, 답변 Reference 저장, AI 호출 로그를 연결해 문서 업로드부터 분석·검색·답변·근거 재조회까지 하나의 서비스 흐름으로 구현했습니다.

기술 스택

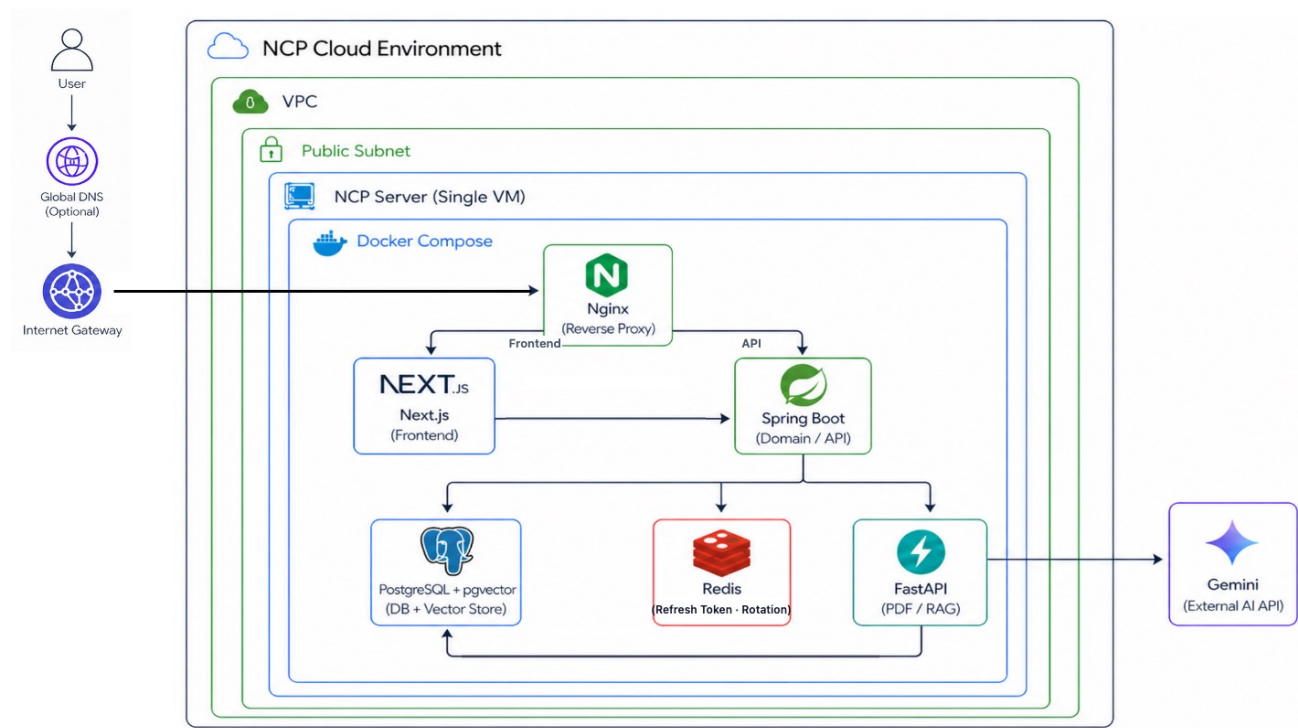
Backend Java · Spring Boot · Python · FastAPI
AI / Data Gemini API · PostgreSQL/pgvector · Redis · LangChain
Frontend Next.js · TypeScript
Infrastructure Docker Compose · Nginx · NCP



운영 화면 · requestId 기반 AiCallLog 조회

NCP 단일 VM에 Docker Compose로 배포한 Runtime 구조

4개 Repository의 애플리케이션과 운영 설정을 Nginx 진입점 아래 하나의 실행 환경으로 조립했습니다.



Repository → Runtime Component Mapping

core-api-spring
Spring Boot · Domain / API

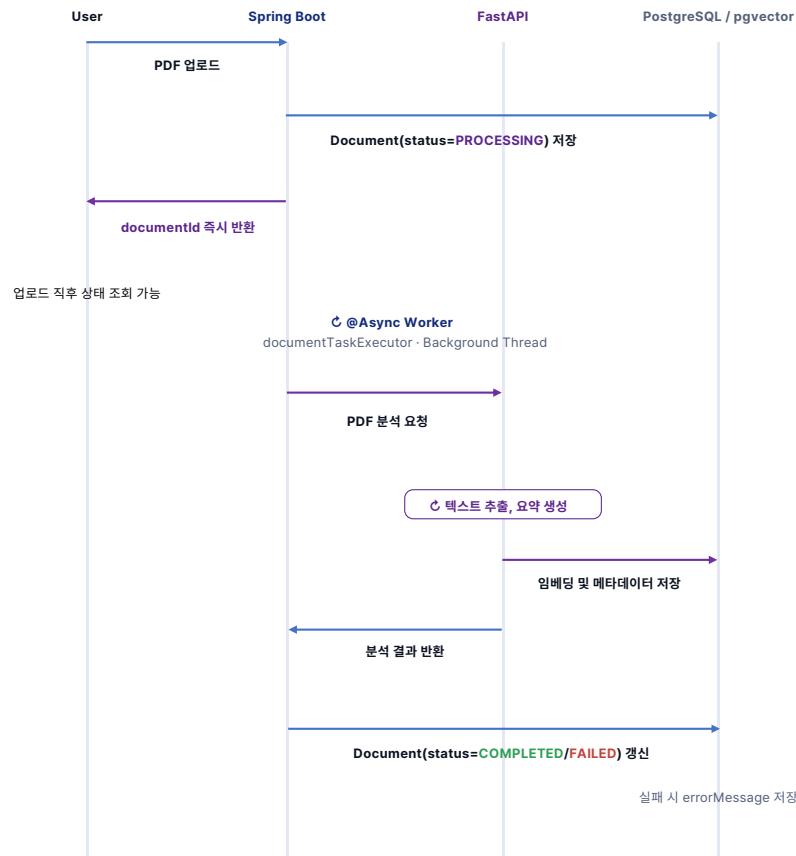
ai-worker-fastapi
PDF · Embedding · Retrieval · LLM

frontend-ui
Next.js · Chat / Reference UI

infra-config
Docker Compose · Nginx · Environment

도메인 상태와 AI 처리를 분리한 비동기 문서 분석 구조

Spring Boot는 서비스 상태와 계약을 관리하고, FastAPI는 문서 분석과 AI 처리를 담당하도록 분리했습니다.



01 배경

PDF 파싱과 Embedding은 처리 시간이 길어, 분석이 끝날 때까지 업로드 요청을 유지하면 사용자 응답과 장시간 AI 처리가 결합되는 문제가 있었습니다.

02 문제

외부 AI API를 기다리는 동안 요청 Thread와 DB Transaction을 장시간 유지하면 응답 지연과 Connection 점유 범위가 커질 수 있었습니다. 또한 처리 실패를 상태로 남기지 않으면 사용자가 문서 분석 결과를 다시 확인하기 어려웠습니다.

03 설계 판단

Spring Boot는 사용자 소유권, Document 상태, API 계약과 결과 저장을 담당하도록 했습니다. FastAPI는 PDF 파싱, Summary, Chunking, Embedding과 Vector 저장을 담당하도록 책임을 분리했습니다. 외부 AI API 대기 구간은 장시간 DB Transaction과 분리하고, 분석 결과를 저장할 때 필요한 DB 작업만 수행했습니다.

04 실제 적용과 결과

Spring의 @Async와 전용 ThreadPoolTaskExecutor를 사용해 Document를 **PROCESSING** 상태로 저장한 뒤 분석 완료를 기다리지 않고 **documentId** 즉시 반환했습니다. 별도 Worker Thread가 FastAPI를 호출하고, 성공 시 **COMPLETED**, 실패 시 **FAILED** 와 errorMessage를 저장했습니다. 업로드 직후 처리 상태 조회가 가능해졌고, **사용자 응답과 AI 처리 시간을 분리** 했으며, 실패 상태를 Document 데이터로 추적할 수 있습니다.

05 트레이드오프와 다음 개선

Thread Pool의 **Core 5 · Max 10 · Queue 50** 은 부하 시험으로 도출한 최적값이 아닌 초기 설정입니다. 동시 업로드가 증가하면 실행 지표를 기반으로 Pool 크기를 조정하고, 작업 Queue와 독립 Worker를 도입해 API 요청과 문서 처리 Lifecycle을 분리할 계획입니다.

Hybrid Retrieval과 답변 근거 저장 구조

pgvector MMR과 키워드 검색을 결합하고, 선택된 문서 근거를 답변과 함께 저장했습니다.

01 Scope Filter

notebook_id · document_id
다른 Notebook의 문서가 검색 결과에 섞이지 않도록 검색 범위를 먼저 제한



02 Hybrid Candidate Search

pgvector Dense MMR

10건 탐색 · 8건 후보 의미 유사도와 후보 다양성 확보

PostgreSQL ILIKE

최대 5건 후보 추가 본문·제목·섹션의 정확 문자열 검색

↘ 합류 ↙



03 Candidate Refinement

Dense·Keyword 후보 병합 → 동일 Chunk 제거 → 규칙 기반 재정렬 → 최종 Context 5건

Dense MMR: 10건 탐색 · 8건 후보 · Keyword: 최대 5건 추가 · MMR 0.5

초기 구현값



04 답변 생성 및 근거 저장

LLM 답변 생성

ChatHistory 1 : N ChatReference

Document ID · Document Title Section Title · Page Number Chunk Index · Page Chunk Index Chunk Content · Sort Order

답변과 선택된 문서 근거를 함께 저장해 새로고침 이후에도 동일한 근거를 다시 조회

01 배경

Vector 유사도 검색은 의미가 비슷한 문서를 찾는 데 유용하지만, 인물명·날짜·숫자처럼 정확한 문자열 조건을 놓칠 수 있었습니다.

02 문제

유사도 결과만 사용하면 비슷한 Chunk가 반복되거나, 질문과 직접 관련된 문장보다 주변 문맥이 우선될 수 있었습니다. 또 한 답변만 저장하면 새로고침 이후 어떤 문서를 근거로 생성했는지 다시 확인하기 어려웠습니다.

03 설계 판단

Notebook과 Document 범위를 제한한 뒤, pgvector MMR 후보와 PostgreSQL ILIKE 후보를 병합했습니다. 중복 Chunk를 제거하고, 질문 토근 중복도·직접 포함 여부·기존 MMR 순위를 조합한 규칙 기반 점수로 재정렬해 최종 Context 를 구성했습니다.

04 실제 적용과 결과

선택된 Chunk는 답변과 함께 ChatReference로 저장했습니다. 문서 제목, Section, Page, Chunk 순서와 원문을 보존해 새로고침 이후에도 동일한 답변 근거를 다시 조회할 수 있도록 했습니다.

05 한계와 다음 개선

현재 재정렬은 고정 규칙 기반이며, 정답 데이터셋을 이용한 검색 품질 평가는 수행하지 않았습니다. 평가 질문과 정답 근거를 구성해 Recall·MRR 등의 지표를 측정하고, 필요하면 학습형 Reranker와 비교할 계획입니다.

Quota 실패 시 부분 저장을 정리하고 호출 원인을 추적한 구조

Embedding을 배치 처리하고, 최종 실패 시 Vector 정리·Document 상태 전이·호출 로그 기록으로 실패 범위를 구분했습니다.

01 Chunk 배치 저장

Chunk 10건씩 배치 저장 · 배치 간 1.5초



02 Quota 오류 감지

429 · RESOURCE_EXHAUSTED

Gemini Embedding API 한도 초과



03 재시도

최대 2회 · 2초 → 4초

Quota 오류에만 Backoff 적용



04 최종 실패 처리

부분 Vector 삭제 시도

데이터 상태

부분 Vector 삭제 시도 → Document FAILED

운영 추적

requestId 연결 → AiCallLog 기록



05 실패 로그 보존

AiCallLog · REQUIRES_NEW

본 요청 실패와 별도 Transaction으로 기록
requestId · 성공 여부 · 응답 시간 · 오류 정보

01 배경

대용량 PDF를 한 번에 Embedding하면 무료 Gemini API의 처리량 제한으로 429 또는 RESOURCE_EXHAUSTED 오류가 발생할 수 있었습니다. 문서를 여러 Batch로 저장하던 중 실패하면 앞선 Batch의 Vector만 남을 수 있었습니다.

02 문제

부분 저장된 Vector가 남으면 Document는 실패했지만 일부 Chunk가 검색 후보로 사용되는 상태 불일치가 발생할 수 있었습니다. 또한 Spring과 FastAPI의 로그가 분리되어 어떤 어떤 요청이 어느 구간에서 실패했는지 연결하기 어려웠습니다.

03 설계 판단

Chunk를 10건씩 배치 처리하고, Quota 오류에만 최대 2회의 Exponential Backoff를 적용했습니다. 재시도 이후에도 실패하면 이미 저장된 Vector를 document_id 기준으로 정리하고, Spring의 Document를 FAILED 상태로 전환하도록 했습니다. Spring과 FastAPI에는 동일한 requestId를 전달하고, AI 호출 결과는 AiCallLog에 별도로 기록했습니다.

04 실제 적용과 결과

일부 Batch가 저장된 뒤 오류가 발생해도 부분 Vector 정리를 시도한 뒤 실패 상태로 종료하도록 구성했습니다. Document 실패와 AI 호출 실패를 구분하고, 관리자 화면에서 요청 유형·성공 여부·응답 시간·오류 내용을 조회할 수 있도록 했습니다. 실패 로그는 REQUIRES_NEW Transaction으로 저장해 본 요청의 Rollback과 분리했습니다.

05 트레이드오프와 다음 개선

Batch 10건, 배치 간 1.5초, 최대 2회 재시도는 부하 시험으로 도출한 최적값이 아니라 무료 Tier 환경을 고려한 초기 운영 설정입니다. Cleanup 자체가 실패하면 부분 Vector가 남을 수 있으므로, 향후에는 삭제 결과 검증과 보상 작업 Queue를 추가해 재처리 가능한 상태로 관리할 필요가 있습니다. 실행 지표가 쌓이면 문서 크기와 오류율을 기준으로 Batch 크기와 재시도 정책을 재조정할 계획입니다.

NotebookLM Clone / 설계 리뷰와 다음 검증

구현한 기능을 나열하기보다, 설계하며 달라진 판단과 검증하지 못한 가정을 정리했습니다.

01 전 프로젝트 대비 개선한 점

1. 책임 분리

Spring Boot는 도메인 상태와 API 계약을, FastAPI는 문서 분석과 AI 처리를 담당하도록 분리했습니다.

2. 처리 상태 관리

장시간 분석을 비동기로 실행하고 PROCESSING · COMPLETED · FAILED 상태로 결과를 추적했습니다.

3. 답변 근거 보존

선택된 Chunk를 ChatReference로 저장해 새로고침 이후에도 답변 근거를 다시 조회할 수 있게 했습니다.

4. 실패 진단 구조

requestId, AiCallLog, 오류 코드와 응답 시간을 연결해 문서 실패와 AI 호출 실패를 구분했습니다.

02 구현하며 얻은 인사이트

1. RAG 품질

모델뿐 아니라 검색 범위, 후보 구성, 재정렬, 근거 저장에 함께 설계되어야 했습니다.

2. 비동기 처리

빠른 응답뿐 아니라 Thread Pool, DB Connection, 외부 API Rate Limit을 함께 측정해야 했습니다.

3. 부분 성공

일부 Vector가 저장된 상태의 실패에는 재시도뿐 아니라 보상 처리와 실패 상태가 필요했습니다.

4. 운영 로그

요청이 Rollback되더라도 실패 원인과 호출 시간은 별도로 남아야 했습니다.

03 구현 당시 선택을 다시 검토

1. Conversation Summary + Recent Window

한계: 독립 질문에도 과거 문맥이 개입할 수 있음

다음: Follow-up 판별과 Query Rewrite 비교

2. Batch 10 / Thread Pool Core 5 · Max 10 · Queue 50

한계: 부하 시험으로 도출한 설정값이 아님

다음: 동시 업로드 · DB Connection · Rate Limit 측정

3. Rule-based Reranker

한계: 설명 가능하지만 검색 품질의 상한이 존재

다음: 고정 평가셋으로 Cross-encoder · LLM Reranker 비교

4. Partial Vector Cleanup

한계: 완전한 원자적 교체 구조는 아님

다음: Staging Table · Job Version 기반 교체 검토

04 남은 한계와 성장 방향

1. 검색 평가 체계

정답 근거가 포함된 평가 질문을 만들고 Recall-MRR-근거 적합도를 측정할 필요가 있습니다.

2. 독립 작업 처리 구조

동시 업로드가 증가하면 API 요청과 문서 처리를 Queue-Worker 구조로 분리해야 합니다.

3. 원자적 Vector 교체

Cleanup 보상 방식에서 Staging-Version 기반 교체 구조를 검토할 필요가 있습니다.

4. 근거 정밀도

Chunk 단위 Reference를 문장·차표 단위로 세분화해 PDF 원문과의 연결성을 높일 수 있습니다.

자연어로 상품을 탐색하고 행동까지 연결하는 AI 쇼핑 서비스

자연어 상품 탐색과 대화형 추천 결과를 장바구니·주문·반품 기능까지 연결했습니다.

기간	형태	역할
2025.10 ~ 2025.11	5인 팀 프로젝트	Frontend 상태 관리 · AI 검색 세션 복원 Guest Cart 병합 · 인증 흐름 개선

GitHub <https://github.com/seochanjin/Accio>

기획 배경

키워드 중심의 쇼핑 검색은 사용자가 원하는 분위기, 용도와 취향을 구체적으로 표현하기 어렵다는 문제에서 시작했습니다. 자연어로 요구사항을 입력하고 대화를 이어가며 상품을 좁혀갈 수 있는 쇼핑 경험을 구현하고자 했습니다.

문제 정의

AI 검색 결과와 대화 상태가 화면 이동이나 새로고침 이후 사라지면 사용자는 탐색을 처음부터 반복해야 했습니다. 또한 비회원 장바구니와 로그인 후 장바구니가 분리되고, 인증 갱신 실패가 사용자 흐름을 끊을 수 있었습니다.

핵심 가설

검색 조건과 AI 대화 상태를 세션 단위로 복원하고, 비회원 상태의 장바구니를 로그인 계정에 병합하면 탐색부터 구매까지의 사용자 흐름을 유지할 수 있다고 판단했습니다.

목표

AI 검색 세션 복원, 비회원 장바구니 병합, Refresh Token 기반 인증 흐름을 적용해 화면 이동·새로고침·로그인 전환 이후에도 사용자가 탐색과 구매를 이어갈 수 있도록 구현했습니다.

기술 스택

Frontend	React · TypeScript
Backend	FastAPI
AI / Search	AWS Bedrock · Titan Embedding · Elasticsearch
Data	MongoDB · Redis
Infrastructure	AWS



메인 화면 · 자연어 상품 검색과 추천 결과

AI 검색 세션 복원

검색 조건과 대화 결과를 세션 단위로 저장하고 화면 이동·새로고침 이후 복원

Guest Cart 병합

로그인 시 비회원 장바구니 항목을 계정 장바구니로 병합해 구매 흐름 유지

인증 흐름 개선

Access Token 만료 시 Refresh Token으로 갱신하고 갱신 실패 시 인증 상태 정리

팀 구현

키워드와 의미 검색을 결합한 상품 탐색 구조

기존 Elasticsearch 키워드 검색에 Titan V2 임베딩 기반 KNN 검색을 결합했습니다.

01 사용자 자연어 질의

“집에서 편하게 입을 따뜻한 옷” · “4만원대 가성비 좋은 무선 이어폰”
상품명뿐 아니라 용도·취향·가격 조건을 포함한 입력



02 Query 준비

규칙 기반 정규화 · **Titan V2 Query Embedding** · 1024차원
Redis Query Cache · Claude Fallback



03 Elasticsearch 단일 Hybrid 요청

Keyword Query

multi_match · term · match_phrase fuzziness · field boost

정확한 상품명·브랜드·카테고리 검색

KNN Query

Titan V2 Embedding k=50 · num_candidates=150

표현이 달라도 의미가 가까운 상품 검색

query + knn을 하나의 Elasticsearch 요청으로 실행



04 점수와 정렬 정책

Keyword function_score — reviewCount · rating · 기존 상품 rank 반영

Relevance: _score desc → rank asc
Popular: reviewCount desc → _score desc → rank asc



05 상품 결과

정확한 키워드 조건과 자연어 의미 조건을 하나의 결과 목록으로 반환

결과 부족 시 재검색

조건: 첫 페이지 요청 · 1차 검색 총 결과 10건 미만
Catalog Query Expansion → 확장 Keyword 생성 → 2차 Keyword 검색 → 결과가 더 많을 때만 교체

01 배경

기존 상품 검색은 Elasticsearch의 키워드 검색을 사용했습니다. AI 검색 기능을 추가하면서 “집에서 편하게 입을 옷”이나 “감성적인 홈 인테리어”처럼 상품명과 정확히 일치하지 않는 자연어 의도를 검색에 반영할 필요가 생겼습니다.

02 문제

키워드 검색은 상품명·브랜드·카테고리처럼 정확한 표현을 찾는 데 유리하지만, 다른 표현으로 설명된 사용자의 취향과 용도를 놓칠 수 있었습니다. 반대로 Vector 검색만 사용하면 정확한 상품명이나 문자열 조건의 영향이 약해질 수 있다고 판단해 두 방식을 함께 유지했습니다.

03 설계 판단

키워드 검색을 대체하지 않고, Titan V2로 생성한 Query Embedding 기반 KNN 검색을 추가했습니다. Keyword Query와 KNN Query를 하나의 Elasticsearch 요청에 함께 전달하고, Elasticsearch의 검색 점수와 서비스 정렬 정책으로 최종 상품 목록을 구성했습니다.

04 실제 적용

Keyword Query에는 multi_match, term, match_phrase, fuzziness와 필드별 Boost를 적용했습니다. 또한 reviewCount, rating, 기존 상품 rank를 function_score에 반영해 서비스 정렬 기준에 활용했습니다. KNN Query에는 Titan V2의 1024차원 Query Embedding을 사용해 표현은 달라도 의미가 가까운 상품을 조화했습니다. 첫 페이지 요청에서 1차 검색 총 결과가 10건 미만이면 상품 카탈로그 기반으로 Query를 확장하고, 확장된 키워드로 2차 Keyword 검색을 수행했습니다.

05 한계와 다음 검증

Keyword-only, Vector-only, Hybrid 검색을 비교하는 고정 평가 데이터셋은 구성하지 못했습니다. RRF는 사용 환경의 기능 제약으로 최종 구조에서 제거했고, 최종 구현은 query와 knn을 하나의 Elasticsearch 요청에 전달하는 Elasticsearch 8.x의 기본 Hybrid 검색 동작을 사용했습니다. 이 방식은 Elasticsearch 버전과 설정에 따라 점수 결합 결과가 달라질 수 있습니다. 향후 자연어 상품 질의와 정답 상품을 포함한 평가셋을 구성해 Recall@K, MRR, nDCG@K, Zero-result Rate를 비교하고, Embedding 호출 시간과 전체 검색 응답 시간도 함께 측정할 필요가 있습니다.

팀 검색 구조 — Elasticsearch Hybrid Search · Titan V2 Embedding · Query Expansion

팀 구현 영역이며 개인 단독 구현이 아님

개인 기여 · Cart / Auth 상태 전환

로그인 전환 과정에서도 장바구니와 인증 상태를 유지한 Frontend 흐름

비회원 장바구니를 로그인 계정에 병합하고, Access Token 만료 시 Refresh Token으로 인증 상태를 복구했습니다.

01 Guest Cart 저장

비로그인 사용자가 상품 추가 → localStorage · cart-storage
productId · quantity · selectedColor · selectedSize · 상품 Snapshot

로그인 전 구매 상태를 브라우저에 유지



02 로그인 성공

사용자 인증 완료 로컬 장바구니 존재 여부 확인 — cart-storage 존재 + 상품 항목 1개 이상

로그인 완료 후 장바구니 병합을 후속 처리



03 병합 Payload 변환

상품 식별 옵션 productId · quantity selectedColor · selectedSize

상품 Snapshot priceSnapshot · nameSnapshot imageSnapshot

Frontend 상태를 Backend Cart API 형식으로 변환



04 서버 장바구니 병합 PUT /api/cart

성공

서버 장바구니 병합 → localStorage 제거 → 동일 상태 재전송 방지

실패

오류 기록 → 로그인 상태는 유지

Cart 병합을 로그인 완료와 분리된 후속 작업으로 처리



05 인증 상태 복구

Access Token 만료·부재 → Refresh Token 확인 → Access Token 재발급 요청

성공: 로그인 상태 유지 · 실패: 로그아웃 처리

Backend에 존재하던 재발급 흐름을 Frontend 인증 상태와 연결

개인 구현 — Guest Cart localStorage 저장 · 로그인 후 서버 장바구니 병합 병합 성공 후 로컬 상태 정리 · Refresh Token 기반 Access Token 재발급

01 배경

비회원 사용자가 상품을 장바구니에 넣은 뒤 로그인하면 로컬 장바구니와 계정 장바구니의 저장 위치가 달라졌습니다.
로그인 전 탐색과 구매 준비가 로그인 이후에도 이어지도록 상태 전환을 처리할 필요가 있었습니다.

02 문제

로그인 이후 서버 장바구니만 조회하면 비회원 상태에서 담은 상품이 사라질 수 있었습니다.
또한 Access Token이 만료됐을 때 Refresh Token이 남아 있어도 Frontend에 재발급 흐름이 없어 인증 상태를 유지하지 못하는 문제가 있었습니다.

03 설계 판단

비회원 장바구니는 localStorage에 유지하고, 로그인 성공 시 Backend Cart API 형식으로 변환해 계정 장바구니에 전달했습니다.
장바구니 병합 실패가 로그인 성공까지 막지 않도록, 병합 작업은 인증 완료 이후의 후속 처리로 분리했습니다.
인증은 Access Token 만료 시 Refresh Token을 이용해 재발급을 시도하도록 연결했습니다.

04 실제 적용과 결과

상품 ID, 수량, 옵션과 상품 Snapshot을 병합 Payload로 구성해 서버에 전달했습니다.
병합 성공 후 localStorage의 Guest Cart를 제거해 동일한 장바구니 상태가 다시 전송되는 것을 막았습니다.
Access Token이 없거나 만료된 경우에는 Refresh Token으로 Access Token 재발급을 요청하고, 성공 시 로그인 상태를 유지하도록 구성했습니다. 병합 오류는 기록하되 로그인 흐름은 계속 진행하도록 처리했습니다.

05 한계와 다음 개선

당시에는 서버와 로컬에 동일 상품이 존재할 때의 수량 합산, 옵션 충돌, 중복 요청 처리 정책을 명시적인 도메인 계약으로 충분히 남기지 못했습니다.
병합 API의 멍등성, Refresh Token 만료·재발급 실패, 네트워크 오류 시 상태 전환을 포함한 통합 테스트도 부족했습니다.
다시 설계한다면 병합 규칙을 별도 도메인 정책으로 분리하고, 로그인·병합 성공·병합 실패·토큰 만료 시나리오를 E2E 테스트로 검증할 것입니다.

ACCIO / 구현 회고와 성장 방향

팀 구현과 개인 기여를 구분하고, 빠른 MVP 개발에서 확인한 한계와 이후 달라진 설계 기준을 정리했습니다.

팀 구현

자연어 상품 탐색 Keyword / Vector 검색 AI Tool 실행 AI 대화·검색 상태 복원

확인 가능한 개인 기여

Guest Cart 로그인 병합 Refresh Token 기반 인증 복구 LoginPage 타입-오류 처리
보완 역할별 타입 인터페이스 분리

검증 근거

GitHub Commit / Diff 서비스 화면 Architecture 기존 프로젝트 문서

01 빠른 통합 과정에서 드러난 문제

1. 상태 관리 복잡도 증가

AI Tool·장바구니·인증을 단기간에 통합하며 화면과 서버의 여러 상태를 함께 관리해야 했습니다.

2. 검색 품질 검증 부족

Keyword-only·Vector-only·Hybrid 검색을 동일한 평가셋으로 비교하지 못했습니다.

3. 책임 범위 불명확성

팀 Search Engine과 개인 기여를 Commit·PR·문서로 명확히 구분하지 못했습니다.

4. 구현 근거 부족

빠른 기능 통합에 비해 설계 판단과 테스트 결과를 충분히 남기지 못했습니다.

02 개인 기여에서 얻은 인사이트

1. 상태 전환

Guest Cart 병합에서는 저장 위치보다 전환 조건과 기준 상태가 중요했습니다.

2. 실패 격리

병합 실패가 로그인 성공까지 막지 않도록 후속 작업으로 분리했습니다.

3. 인증 복구

Access Token 만료와 Refresh Token 재발급을 하나의 인증 Lifecycle로 관리했습니다.

4. 상태 정리

병합 성공 후 로컬 상태를 제거해 동일한 장바구니 상태의 재전송을 막았습니다.

03 남은 한계와 다음 검증

1. Cart 병합 계약

수량 합산·옵션 충돌·중복 요청 정책을 명시적인 도메인 계약으로 남기지 못했습니다.

2. 역동성과 실패 복구

병합 요청을 재시도했을 때 동일한 결과가 보장되는지 검증하지 못했습니다.

3. 인증 시나리오

Refresh Token 만료·재발급 실패·네트워크 오류를 포함한 E2E 테스트가 부족했습니다.

4. 검색 품질 평가

Keyword-only·Vector-only·Hybrid 검색을 고정 평가셋으로 비교하지 못했습니다.

04 개인 고민과 성장 방향

1. 기능 구현과 Ownership

기능을 만드는 것과 내 판단·수정·검증 범위를 근거로 남기는 것은 별개의 문제였습니다.

2. AI 기능과 Backend 설계

AI 연결 자체보다 상태 전환·실패 처리·권한·데이터 계약 설계가 더 중요했습니다.

3. 기술 방향

이후에는 AI 기능을 안정적인 서비스 흐름으로 연결하는 AI Backend와 Platform 영역에 집중했습니다.

4. 후속 프로젝트 반영

NotebookLM과 NewsLab에서는 책임 분리·명시적 상태·실패 기록·운영 검증을 개발 과정에 포함했습니다.